

PC5

下面继续讲 PC5.pdf: 第五章 并行算法的一般设计方法。这章属于第二篇“并行算法的设计”，重点讲三种常用的并行算法设计思路：**直接并行化串行算法、从问题本身重新设计并行算法、借用已有算法解决新问题。**

1. 第五章总体结构

本章标题是 **并行算法的一般设计方法**。

它包括三节：

1.1 5.1 串行算法的直接并行化

就是从已有串行算法出发，分析其中哪些部分可以同时执行，然后把它改造成并行算法。

1.2 5.2 从问题描述开始设计并行算法

不依赖已有串行算法，而是从问题本身的结构、性质和约束出发，重新设计一个并行算法。

1.3 5.3 借用已有算法求解新问题

找出当前问题和某个已解决问题之间的联系，把已有算法改造后用于新问题。

这三种方法可以理解为并行算法设计的三个层次：

已有串行算法 → 直接并行化
问题本身特性 → 重新设计并行算法
问题之间联系 → 借用成熟算法思想

2. 5.1 串行算法的直接并行化

2.1 设计方法描述

这一方法的核心是：

■ 发掘和利用现有串行算法中的并行性，直接将串行算法改造为并行算法。

也就是说，先有一个串行算法，然后分析它内部哪些步骤可以并发执行。

例如：

```
for i = 1 to n:  
    C[i] = A[i] + B[i]
```

这个循环中每次迭代互不依赖，因此可以直接改成：

```
for i = 1 to n par-do:  
    C[i] = A[i] + B[i]
```

这就是最典型的直接并行化。

2.2 方法评注

课件第 5 页给出四点评注。

2.2.1 这是最常用的方法之一

因为现实中很多程序已经有成熟的串行版本。

直接并行化可以复用原来的算法思想、数据结构和程序框架，工程成本较低。

尤其是数值计算程序，例如矩阵运算、网格计算、有限差分、积分求和等，经常可以从循环并行化入手。

2.2.2 不是所有串行算法都能直接并行化

如果串行算法中存在强依赖，就很难并行化。

例如：

```
x[1] = f(x[0])
x[2] = f(x[1])
x[3] = f(x[2])
...
```

这里每一步都依赖前一步结果，天然串行性很强。

这种算法即使强行并行化，也可能通信和同步开销很大，得不到好效果。

2.2.3 好的串行算法不一定能变成好的并行算法

这是本节非常重要的一句话。

串行算法关注的是：

总操作数少
存储少
控制简单

并行算法还要关注：

并发度高
通信少
同步少
负载均衡
可扩展

所以一个串行最优算法，可能因为依赖太强、任务太不均衡或通信太复杂，而不是好的并行算法。

2.2.4 很多数值串行算法可以并行化为有效并行算法

数值算法通常有规则数据结构，例如向量、矩阵、网格。

这些结构容易划分，也容易映射到处理器上。

例如：

- 矩阵加法
- 矩阵乘法
- Jacobi 迭代
- FFT
- 向量内积
- 数值积分

这些问题通常具有明显的数据并行性。

3. 5.1.2 快排序算法的并行化

课件第 7 页给出 算法 5.2: PRAM-CRCW 上的快排序二叉树构造算法。

3.1 算法目标

输入：

序列 $(A_1, A_2, \dots, A_n)$
 n 个处理器

输出：

供排序用的一棵二叉排序树

这个算法不是直接把快速排序的递归过程简单分给多个处理器，而是构造一棵二叉排序树。
构造完成后，对这棵树进行中序遍历，就可以得到有序序列。

3.2 为什么叫快排序的并行化

普通快速排序的思想是：

```
选一个基准元素 pivot
小于 pivot 的放左边
大于 pivot 的放右边
然后递归处理左右子序列
```

二叉排序树的插入过程也类似：

```
比当前节点小 → 走左子树
比当前节点大 → 走右子树
```

所以用二叉排序树构造来表达快速排序的分裂思想，是一种自然的并行化方式。

3.3 PRAM-CRCW 的作用

该算法运行在 **PRAM-CRCW** 模型上。

CRCW 表示：

```
Concurrent Read Concurrent Write
并发读，并发写
```

也就是说，多个处理器可以同时读同一个共享变量，也可以同时尝试写同一个共享变量。

这对二叉树并行构造很重要，因为多个元素可能同时竞争某个节点的左孩子或右孩子位置。

3.4 算法变量含义

课件中的变量大致可以这样理解：

变量	含义
<code>Ai</code>	第 i 个待排序元素
<code>root</code>	当前树根
<code>fi</code>	处理器 i 当前比较的父节点
<code>LCi</code>	节点 i 的左孩子
<code>RCi</code>	节点 i 的右孩子
<code>n+1</code>	空指针或空节点标记

3.5 初始化阶段

算法第一部分是：

```
for each processor i do
  root = i
  fi = root
  LCi = RCi = n + 1
end for
```

这一步为每个元素初始化二叉树相关指针。

其中 `LCi` 和 `RCi` 初始化为空，表示节点 i 暂时没有左右孩子。

3.6 并行插入阶段

第二部分是重复执行，直到每个非根节点插入成功。

判断条件是：

```
if (Ai < Afi) or (Ai = Afi and i < fi)
```

意思是：

- 如果 A_i 小于当前父节点 A_{fi} ，就应该进入左子树；
- 如果两个元素相等，则用下标 $i < fi$ 打破平局，保证排序稳定或至少保证树结构确定。

如果应进入左子树，则执行：

```
LCfi = i
if i = LCfi then exit
else fi = LCfi
```

如果写入成功，说明节点 i 成为当前父节点的左孩子，可以退出。

如果没有成功，说明可能有竞争或该位置已有节点，就继续沿左孩子方向向下找。

右子树逻辑类似：

```
RCfi = i
if i = RCfi then exit
else fi = RCfi
```

3.7 算法思想总结

这个并行快排算法的本质是：

每个元素作为一个处理器任务
所有元素同时尝试插入二叉排序树
通过比较决定走左子树还是右子树
利用 PRAM-CRCW 处理并发访问
最终形成一棵二叉排序树

构造完成后，对树做中序遍历即可得到排序结果。

3.8 这个例子说明了什么

它说明“直接并行化串行算法”并不一定是简单地把循环变成并行循环。

对于快速排序这种递归分治算法，并行化可以有不同方式：

方式一：左右子问题递归并行
方式二：并行 partition
方式三：构造并行二叉排序树

课件这里展示的是第三种：把快速排序思想转化为并行树构造。

4. 5.2 从问题描述开始设计并行算法

4.1 方法描述

这一方法的核心是：

从问题本身描述出发，不考虑相应的串行算法，设计一个全新的并行算法。

也就是说，设计者暂时不问：

这个问题串行算法怎么做？

而是直接问：

这个问题本身有哪些结构？
哪些操作可以同时做？
数据之间有什么独立性？
问题是否有周期性、对称性、局部性、递归性？

4.2 与直接并行化的区别

直接并行化是：

已有串行算法 → 找并行性 → 改造成并行算法

从问题描述开始设计是：

问题本身 → 挖掘结构 → 重新设计并行算法

后者更难，但可能得到更好的并行算法。

4.3 方法评注

课件第 9 页给出三点评注。

4.3.1 要挖掘问题固有特性与并行性的关系

不同问题有不同的内在结构。

例如：

- 矩阵问题有块结构；
- 图问题有邻接结构；
- 字符串问题有周期性；
- 网格问题有局部邻域；
- 排序问题有比较关系；
- FFT 有蝶形递归结构。

好的并行算法往往不是机械改造串行算法，而是抓住这些结构。

4.3.2 设计全新并行算法具有挑战性和创造性

这种方法要求设计者有较强的算法背景和抽象能力。

因为没有现成串行流程可以照搬，需要重新设计：

数据如何划分
任务如何定义
并行步骤如何组织
通信如何安排
同步在哪里发生
复杂度如何分析

4.3.3 串周期性的 PRAM-CRCW 算法是典型范例

课件提到，利用字符串周期性设计 PRAM-CRCW 算法是一个好例子。

这里的意思是：

字符串匹配问题不一定要照搬串行 KMP 或 Boyer-Moore，而可以从字符串自身的周期性出发，设计适合并行模型的算法。

4.4 这种方法的优缺点

4.4.1 优点

可能得到高度并行、复杂度更低的算法。

特别是在 PRAM、BSP 等理论模型下，从问题结构直接设计，往往能得到漂亮的复杂度结果。

4.4.2 缺点

难度大，不容易系统化。

很多时候需要灵感和经验，不像“直接并行化”那样有比较明确的操作路径。

5. 5.3 借用已有算法求解新问题

5.1 方法描述

这一方法的核心是：

找出求解问题和某个已解决问题之间的联系，改造或利用已知算法应用到求解问题上。

也就是说，不是完全从零开始，而是寻找问题之间的结构相似性。

5.2 方法逻辑

可以概括为：

```
新问题 A
↓ 找联系
已知问题 B
↓ 借用 B 的算法框架
改造成 A 的并行算法
```

这类方法在算法设计中非常常见。

例如：

- 用矩阵乘法求传递闭包；
- 用矩阵乘法求所有点对最短路径；
- 用排序解决几何问题；
- 用图遍历解决连通性问题；
- 用前缀和解决压缩、扫描、编号问题。

5.3 方法评注

课件第 12 页指出：

5.3.1 这是一项创造性工作

因为关键在于发现两个问题之间的隐藏联系。

表面看起来不同的问题，可能有相同的代数结构或递推形式。

5.3.2 矩阵乘法求所有点对最短路径是典型范例

课件后面就讲这个例子。

它的核心思想是：

最短路径问题可以转化成一种特殊意义下的“矩阵乘法”。

6. 5.3.2 利用矩阵乘法求所有点对间最短路径

课件第 14 页讲 **利用矩阵乘法求所有点对间最短路径**。

这是本章最重要的例子。

6.1 问题描述

给定一个有向图：

$$G = (V, E)$$

边权矩阵为：

$$W = (w_{ij})_{n \times n}$$

要求最短路径长度矩阵：

$$D = (d_{ij})_{n \times n}$$

其中 d_{ij} 表示从顶点 v_i 到顶点 v_j 的最短路径长度。

课件假定图中没有负权有向回路。

这个假设很重要，因为如果有负权回路，最短路径可能无限小，不再有良好定义。

6.2 初始矩阵 D^1

定义：

$$d_{ij}^{(1)} = w_{ij}, \text{ 当 } i \neq j$$

如果 v_i 到 v_j 之间没有边，则记为：

∞

同时：

$$d_{ii}^{(1)} = 0$$

因为从一个顶点到自身的距离为 0。

所以 $D^{(1)}$ 就是初始边权矩阵，也就是只允许使用直接边时的最短路径矩阵。

6.3 最短路径与中间节点数

课件定义：

$$d_{ij}^{(k)}$$

表示从 v_i 到 v_j ，至多有 $k-1$ 个中间节点的最短路径长度。

也就是说：

- $D^{(1)}$ ：只允许 0 个中间节点，即直接边；
- $D^{(2)}$ ：允许至多 1 个中间节点；
- $D^{(3)}$ ：允许至多 2 个中间节点；
- 以此类推。

如果图中没有负权回路，则最终：

$$d_{ij} = d_{ij}^{(n-1)}$$

因为一个 n 个顶点的简单最短路径最多经过 $n-1$ 条边，不需要重复顶点。

6.4 最优性原理

课件给出递推公式：

$$d_{ij}^{(k)} = \min_{1 \leq l \leq n} \{ d_{il}^{(k/2)} + d_{lj}^{(k/2)} \}$$

这句话非常关键。

它表示：

从 i 到 j 、允许较多中间点的最短路径，可以拆成两段：

$$\begin{aligned} i &\rightarrow l \\ l &\rightarrow j \end{aligned}$$

其中每一段都只需要允许一半规模的中间点。

然后在所有可能中转点 l 中取最小值。

6.5 与矩阵乘法的类比

普通矩阵乘法是：

$$C_{ij} = \sum_l A_{il} \times B_{lj}$$

而最短路径递推是：

$$d_{ij} = \min_l \{ d_{il} + d_{lj} \}$$

课件指出，可以把：

普通乘法中的 $\times$ 看作 $+$
普通乘法中的 $\sum$ 看作 $\min$

也就是在一个特殊代数系统中做“矩阵乘法”。

更准确地说，这是 **min-plus 矩阵乘法**。

普通矩阵乘法：

$$C_{ij} = \sum_l A_{il} \times B_{lj}$$

最短路径矩阵乘法：

$$C_{ij} = \min_l \{ A_{il} + B_{lj} \}$$

所以所有点对最短路径可以借用矩阵乘法的思想求解。

7. $D^1 \rightarrow D^2 \rightarrow D^4 \rightarrow \dots$ 的倍增过程

7.1 递推过程

课件给出：

$$D^1 \rightarrow D^2 \rightarrow D^4 \rightarrow \dots \rightarrow D^{2^{\lceil \log n \rceil}} = D^n$$

每一步都相当于做一次 min-plus 矩阵乘法：

$$\begin{aligned} D^2 &= D^1 \otimes D^1 \\ D^4 &= D^2 \otimes D^2 \\ D^8 &= D^4 \otimes D^4 \\ &\dots \end{aligned}$$

其中 $\otimes$ 表示 min-plus 意义下的矩阵乘法。

7.2 为什么是倍增

因为每次计算 D^k 时，都由两个 $D^{\lceil k/2 \rceil}$ 合成。

这类似并行算法中的 doubling technique：

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots \rightarrow n$$

经过 $O(\log n)$ 轮，就能覆盖最多 $n-1$ 个中间节点的路径。

7.3 为什么能用并行矩阵乘法

每个 d_{ij} 的计算是：

$$\min_l \{ d_{il} + d_{lj} \}$$

对于不同的 (i, j) ，可以并行计算。

对于同一个 (i, j) ，不同的 l 也可以并行计算后做归约取最小。

所以这个问题天然适合并行化。

8. 这个例子的算法意义

8.1 它不是直接使用 Floyd 算法

所有点对最短路径最经典的串行算法是 Floyd-Warshall：

```
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      d[i][j] = min(d[i][j], d[i][k] + d[k][j])
```

但课件这里不是直接并行化 Floyd，而是借用矩阵乘法思想。

8.2 它体现了“借用已有算法”的方法

本来要求解的是图论问题：

所有点对最短路径

但通过分析递推结构，发现它和矩阵乘法很像。

于是可以把它转化为：

min-plus 矩阵乘法反复平方

这就是典型的“借用已有算法求解新问题”。

8.3 它适合并行计算

矩阵乘法是并行计算中研究非常充分的问题。

如果一个问题能转化为矩阵乘法，就可以借用已有的并行矩阵乘法算法、复杂度分析和硬件映射方式。

9. 三种设计方法的对比

方法	出发点	优点	缺点	例子
串行算法直接并行化	已有串行算法	简单、常用、工程成本低	不一定得到好并行算法	快排序并行化
从问题描述开始设计	问题本身结构	可能得到更高效并行算法	难度大、创造性强	利用串周期性设计 PRAM 算法
借用已有算法	问题之间联系	可复用成熟算法思想	需要发现深层结构相似性	矩阵乘法求所有点对最短路径

10. 本章核心总结

第五章讲的是并行算法设计的三条基本路线。

10.1 第一条路线：改造已有串行算法

适合已有成熟算法，而且算法内部存在明显并行性的情况。

典型问题：

循环独立
数据块独立
递归子问题独立
数值计算规则

10.2 第二条路线：重新设计并行算法

适合串行算法并行性差，或者问题本身有更强并行结构的情况。

这种方法更理论化，也更有创造性。

10.3 第三条路线：借用已有算法

适合能找到问题之间代数结构、递推结构或图结构相似性的情况。

本章例子中，最短路径问题被转化为 min-plus 矩阵乘法，就是典型案例。

11. 复习重点

11.1 必须掌握的概念

本章最重要的是理解三种方法：

- 5.1 串行算法的直接并行化
- 5.2 从问题描述开始设计并行算法
- 5.3 借用已有算法求解新问题

11.2 必须理解的判断

一个好的串行算法不一定能变成好的并行算法。

原因是并行算法还要考虑：

并发度
通信开销
同步开销
负载均衡
处理器数量
存储分布
模型适配性

11.3 必须理解的例子

利用矩阵乘法求所有点对最短路径的关键是：

普通矩阵乘法：
 $C_{ij} = \sum_l A_{il} \times B_{lj}$

最短路径：
 $d_{ij} = \min_l \{ d_{il} + d_{lj} \}$

对应替换关系是：

$\times \rightarrow +$
 $\Sigma \rightarrow \min$

因此可以借用矩阵乘法思想，通过：

$D^1 \rightarrow D^2 \rightarrow D^4 \rightarrow \dots \rightarrow D^n$

逐步得到所有点对最短路径。

11.4 一句话总结

本章可以概括为：

并行算法设计既可以从串行算法出发，也可以从问题本身出发，还可以从已知问题和算法中借力；关键是找到适合并行执行的结构。

PC6

1 本章结构

第六章包括：

6.1 划分设计技术
6.2 分治设计技术
6.3 平衡树设计技术
6.4 倍增设计技术
6.5 流水线设计技术

可以这样理解：

技术	核心思想	典型应用
划分	把数据或任务切成若干块	排序、归并、选择
分治	分解子问题，并行递归求解	双调归并、排序网络
平衡树	按树形结构逐层归约或传播	最大值、前缀和
倍增	每一步处理距离翻倍	链表排名、求森林根
流水线	把算法流程分成连续阶段	DFT、脉动算法

2.6.1 划分设计技术

划分设计技术的核心是：

把输入数据或计算任务划分成若干部分，使不同处理器可以同时处理不同部分。

课件分为四种划分：

- 6.1.1 均匀划分技术
- 6.1.2 方根划分技术
- 6.1.3 对数划分技术
- 6.1.4 功能划分技术

2.1 6.1.1 均匀划分技术

2.1.1 划分方法

均匀划分就是把 n 个元素平均分成 p 组。

第 i 个处理器负责：

$A[(i-1)n/p + 1 \dots in/p]$

也就是说，每个处理器处理大约 n/p 个元素。

这种划分最直观，也最常见。

2.1.2 适用场景

均匀划分适合：

- 数据规模规则；
- 每个元素处理代价差不多；
- 各处理器工作量容易均衡；
- 通信关系比较简单。

例如数组加法、矩阵分块、排序中的初始分段，都可以用均匀划分。

2.2 PSRS 排序

课件用 **PSRS 排序** 作为均匀划分的例子。

PSRS 全称可以理解为：

Parallel Sorting by Regular Sampling
并行正则采样排序

它运行在 MIMD-SM 模型上。

2.2.1 算法步骤

输入 n 个元素，使用 p 个处理器。

步骤如下：

1. 均匀划分
2. 局部排序
3. 选取样本
4. 样本排序
5. 选择主元
6. 主元划分
7. 全局交换
8. 归并排序

2.2.2 第一步：均匀划分

把数组 $A[1..n]$ 分成 p 段。

第 p_i 个处理器负责：

$A[(i-1)n/p + 1 \dots in/p]$

这样每个处理器拿到一段局部数据。

2.2.3 第二步：局部排序

每个处理器对自己负责的局部段调用串行排序算法。

例如：

P_1 排第 1 段
 P_2 排第 2 段
...
 P_p 排第 p 段

排序后，每个局部段内部有序。

2.2.4 第三步：选取样本

每个处理器从自己的有序子序列中选取 p 个样本元素。

因为每段已经有序，所以可以按固定间隔选样本，这叫 **正则采样**。

样本的作用是估计全局数据分布。

2.2.5 第四步：样本排序

所有处理器共选出：

$p \times p = p^2$

个样本。

课件中用一台处理器对这 p^2 个样本进行串行排序。

2.2.6 第五步：选择主元

从排好序的样本序列中选择 $p-1$ 个主元。

这些主元被广播给所有处理器。

主元的作用是把全局数据范围划成 p 个区间。

例如有 3 个处理器，就选择 2 个主元，把数据划成：

小于主元1
主元1 到 主元2
大于主元2

2.2.7 第六步：主元划分

每个处理器根据主元，把自己的有序局部段划成 p 段。

也就是说，每个处理器都会产生 p 个小块：

属于第 1 个全局区间的元素
属于第 2 个全局区间的元素
...
属于第 p 个全局区间的元素

2.2.8 第七步：全局交换

各处理器把对应区间的数据发送给对应处理器。

例如：

```
所有属于第 1 区间的数据 → P1
所有属于第 2 区间的数据 → P2
...
所有属于第 p 区间的数据 → Pp
```

交换后，每个处理器拿到的是自己负责的全局值域范围内的数据。

2.2.9 第八步：归并排序

每个处理器收到来自其他处理器的若干有序段。

这些段内部已经有序，所以处理器只需要进行归并排序。

最终：

```
P1 的数据 ≤ P2 的数据 ≤ ... ≤ Pp 的数据
```

把各处理器结果拼起来，就是全局有序序列。

2.3 PSRS 例子：N = 27，p = 3

课件第 6 页给了一个完整过程。

2.3.1 初始数据

有 27 个数，使用 3 个处理器。

先均匀划分成 3 段，每段 9 个元素。

2.3.2 局部排序

每个处理器把自己的 9 个数排好序。

局部排序后，三个局部段分别有序，但全局还没有有序。

2.3.3 正则采样

每个处理器从自己的有序段中选择样本。

图中样本包括类似：

```
6, 39, 72
12, 40, 69
20, 33, 72
```

然后把这些样本集中起来排序。

2.3.4 选择主元

样本排序后，从中选出两个主元。

图中主元大致是：

```
33, 69
```

于是全局被划分成三个区间：

```
≤ 33
33 到 69
≥ 69
```

2.3.5 主元划分与全局交换

每个处理器根据 33 和 69 切分自己的有序段，然后把对应区间发送给对应处理器。

2.3.6 最终归并

全局交换后，各处理器分别归并自己的数据，得到最终有序序列：

```
6 12 14 15 20 21 27 32 33
36 39 40 46 48 53 54 58 61 69
72 72 84 89 91 93 97 97
```

这个例子说明 PSRS 的关键不是简单“各排各的”，而是通过采样和主元让每个处理器最终负责一个全局值域范围。

3. 6.1.2 方根划分技术

3.1 划分方法

方根划分是把 n 个元素按长度约为：

$$\sqrt{n}$$

的块来划分。

也就是：

$$A[(i-1)n^{1/2}+1 \dots i n^{1/2}]$$

其中：

$$i = 1 \sim n^{1/2}$$

它的特点是块大小和块数都约为 $\sqrt{n}$ ，适合某些并行递归算法。

3.2 Valiant 归并算法

课件用 **Valiant 归并** 作为方根划分的例子。

问题是归并两个有序数组：

```
A[1..p]
B[1..q]
```

假设：

$$p \leq q$$

处理器数：

$$k = \lfloor \sqrt{pq} \rfloor$$

目标是把两个有序序列并成一个有序序列。

3.3 算法思想

3.3.1 方根划分

把 **A** 和 **B** 分别按方根规模分成若干段。

大致是：

- A 按 $\sqrt{p}$ 分段
- B 按 $\sqrt{q}$ 分段

然后选择划分元。

3.3.2 段间比较

用 A 的划分元和 B 的划分元比较。

目的是判断：

A 的某个划分元应该插入 B 的哪一段

这样可以把大归并问题分解成多个较小的归并问题。

3.3.3 段内比较

当知道某个 A 的划分元落入 B 的某一段之后，再在该段内部进一步比较，确定精确插入位置。

3.3.4 递归归并

插入划分元后，B 被重新分段。

每个 A 的对应段和 B 的对应段形成一对子问题。

对每对子问题递归执行上述过程，直到 A 组为空。

3.4 方根划分例子

课件给出的例子是：

A = {1, 3, 8, 9, 11, 13, 15, 16}, p = 8
B = {2, 4, 5, 6, 7, 10, 12, 14, 17}, q = 9

3.4.1 选取划分元

因为：

$\sqrt{8} \approx 3$
 $\sqrt{9} = 3$

所以 A 中选出类似：

8, 13

B 中选出类似：

5, 10, 17

作为划分元。

3.4.2 比较并重新分段

通过比较，知道 8 应该插入 B 中 7 和 10 之间，13 应该插入 12 和 14 之间。

于是形成若干较小的归并段：

A1: 1, 3
B1: 2, 4, 5, 6, 7, 8

A2: 9, 11
B2: 10, 12, 13

A3: 15, 16
B3: 14, 17

然后对每个小段继续递归。

3.4.3 最终结果

归并后得到：

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

3.5 Valiant 归并算法分析

3.5.1 处理器数

课件说明，在整个并行递归过程中，所需处理器数不超过：

$$k = \lfloor \sqrt{pq} \rfloor$$

原因包括：

- 段间比较对数不超过 $\sqrt{p} \cdot \sqrt{q} = \sqrt{pq}$ ；
- 段内比较也不超过 $\sqrt{pq}$ ；
- 递归子问题的处理器总需求通过 Cauchy 不等式也可限制在 $\sqrt{pq}$ 以内。

所以整体可以用：

$$k = \lfloor \sqrt{pq} \rfloor$$

个处理器完成。

3.5.2 时间复杂度

设 λ_i 是第 i 次递归后 A 组的最大长度。

每次递归后，最大规模大约按平方根下降。

所以达到常数规模只需要：

$$O(\log \log p)$$

轮。

课件结论：

$$t_k(p, q) = O(\log \log p), \quad p \leq q$$

这说明方根划分是一种非常高效的并行归并技术。

4. 6.1.3 对数划分技术

4.1 划分方法

对数划分把 n 个元素按长度约为：

$$\log n$$

的块划分。

即：

$$A[(i-1)\log n + 1 \dots i \log n]$$

其中：

$$i = 1 \sim n/\log n$$

它适合把大问题拆成较小但不是太小的块，减少通信和调度开销。

4.2 对数划分并行归并排序

课件例子是 PRAM-CREW 上的对数划分并行归并排序。

问题：给定两个有序序列：

```
A[1..n]
B[1..m]
```

要归并成一个有序序列。

4.3 rank 操作

课件定义：

```
j[i] = rank(b_{i \log m} : A)
```

意思是：

$b_{i \log m}$ 在 A 中的位序，即 A 中小于等于它的元素个数。

通过 rank，可以确定 B 中某些分界元素在 A 中的位置，从而把归并问题划分成多个独立的小归并问题。

4.4 例子

课件给出：

```
A = (4, 6, 7, 10, 12, 15, 18, 20)
B = (3, 9, 16, 21)
```

其中：

```
n = 8
m = 4
\log m = \log 4 = 2
```

取 B 中每隔 $\log m$ 的元素作为划分点：

```
b2 = 9
b4 = 21
```

计算：

```
j[1] = rank(9 : A) = 3
```

因为 A 中小于等于 9 的元素是：

```
4, 6, 7
```

所以 $\text{rank} = 3$ 。

另一个划分点 21 大于 A 中所有元素，所以对应 rank 为 8。

于是：

```
B0: 3, 9
B1: 16, 21

A0: 4, 6, 7
A1: 10, 12, 15, 18, 20
```

原来的归并问题变成两个较小归并问题：

```
(A0, B0)
(A1, B1)
```

它们可以并行进行。

5. 6.1.4 功能划分技术

5.1 划分方法

功能划分不是简单按连续位置切分，而是：

把 n 个元素分成等长的 p 组，每组满足某种特性。

这里的“功能”指划分出来的组要服务于某个特定算法目标。

5.2 m-n 选择问题

课件用 (m, n) 选择问题 作为例子。

问题是：

从 n 个元素中选出前 m 个最小者

也就是找出最小的 m 个元素。

5.3 功能划分要求

课件指出：

每组元素个数必须大于 m

算法中将 A 划分为：

$g = n/m$

组，每组含 m 个元素。

5.4 算法步骤

输入：

$A = (a_1, \dots, a_n)$

输出：

前 m 个最小者

算法：

- 1. 功能划分：将 A 划分成 $g = n/m$ 组，每组 m 个元素；
- 2. 局部排序：使用 **Batcher** 排序网络将各组并行排序；
- 3. 两两比较：将排好序的各组两两比较，形成 **MIN** 序列；
- 4. 排序-比较：对各个 **MIN** 序列重复排序和比较，直到选出 m 个最小者。

5.5 图 6.3 的含义

第 15 页图 6.3 展示了 $(m-n)$ 选择过程。

图中每两个已排序组进行比较，产生：

MIN 序列
MAX 序列

因为我们只关心前 m 个最小者，所以每次比较后保留更可能包含最小元素的 **MIN** 序列，逐步淘汰不可能进入前 m 的元素。

6. 6.2 分治设计技术

6.1 并行分治设计步骤

分治设计技术的基本步骤是：

1. 将输入划分成若干个规模相等的子问题；
2. 同时递归求解这些子问题；
3. 并行归并子问题的解，直到得到原问题的解。

和串行分治相比，并行分治的关键是：

子问题要并行递归求解
合并过程也要尽量并行

否则只能加速分解部分，合并阶段仍会成为瓶颈。

6.2 双调归并网络

课件用 **双调归并网络** 作为分治技术的例子。

6.3 双调序列

6.3.1 定义直观理解

双调序列是指序列先单调上升再单调下降，或者经过循环移位后具有这种性质。

课件给出的例子：

(1, 3, 5, 7, 8, 6, 4, 2, 0)
(8, 7, 6, 4, 2, 0, 1, 3, 5)
(1, 2, 3, 4, 5, 6, 7, 8)

这些都是双调序列。

其中第三个完全递增序列也可以看成特殊的双调序列。

6.4 Batcher 定理

课件给出 Batcher 定理。

给定双调序列：

$(x_0, x_1, \dots, x_{n-1})$

对每个：

$i = 0, 1, \dots, n/2 - 1$

计算：

$s_i = \min\{x_i, x_{i+n/2}\}$
 $l_i = \max\{x_i, x_{i+n/2}\}$

则：

小序列 $S = (s_0, s_1, \dots, s_{n/2-1})$
大序列 $L = (l_0, l_1, \dots, l_{n/2-1})$

仍然是双调序列。

这个定理是双调归并网络正确性的基础。

6.5 双调归并过程

6.5.1 第一步：两两比较

把序列前半部分和后半部分对应元素比较。

较小者进入 `MIN` 子序列，较大者进入 `MAX` 子序列。

6.5.2 第二步：递归归并

根据 Batcher 定理，`MIN` 和 `MAX` 仍是双调序列。

因此可以递归对 `MIN` 和 `MAX` 做双调归并。

6.5.3 第三步：拼接输出

最终输出：

归并后的 `MIN` 序列 + 归并后的 `MAX` 序列

得到非降有序序列。

6.6 Batcher 双调归并算法

课件给出的伪代码可以概括为：

```
Procedure BITONIC_MERG(X)
输入：双调序列 X = (x0, x1, ..., x_{n-1})
输出：非降有序序列 Y

1. for i = 0 to n/2 - 1 par-do
    s_i = min{x_i, x_{i+n/2}}
    l_i = max{x_i, x_{i+n/2}}
end for

2. 递归调用：
   BITONIC_MERG(S)
   BITONIC_MERG(L)

3. 输出 S followed by L
```

这是一个典型的并行分治算法：

先并行比较
再递归处理两个子问题
最后拼接结果

7. 6.3 平衡树设计技术

7.1 设计思想

平衡树设计技术的核心是：

以树的叶结点为输入，中间结点为处理结点，由叶向根或由根向叶逐层进行并行处理。

这类技术特别适合：

- 最大值；
- 最小值；
- 求和；
- 归约；
- 前缀和；
- 广播。

7.2 两种方向

7.2.1 由叶到根

用于归约问题。

例如：

```
求最大值
求总和
求乘积
```

每一层把两个子节点结果合并到父节点。

7.2.2 由根到叶

用于传播问题。

例如：

```
广播前缀信息
计算前缀和中的反向传播
```

8. 6.3.2 求最大值

8.1 算法思想

输入 n 个元素，把它们放在二叉树叶子上。

每个内部节点计算两个孩子中的最大值。

经过 $\lceil \log n \rceil$ 层后，根节点就是全局最大值。

8.2 算法 6.8

课件给出的算法运行在 SIMD-TC(SM) 上。

伪代码：

```
Begin
  for k = m-1 to 0 do
    for j = 2^k to 2^{k+1}-1 par-do
      A[j] = max{A[2j], A[2j+1]}
    end for
  end for
End
```

其中：

```
n = 2^m
```

8.3 图示理解

第 27 页图中，叶子层是原始输入元素。

倒数第一层处理器两两比较：

```
max(A_n, A_{n+1})
max(A_{n+2}, A_{n+3})
...
```

上一层继续比较局部最大值。

最后根节点得到全局最大值。

8.4 时间和处理器数

课件给出：

$$\begin{aligned}t(n) &= m \times O(1) = O(\log n) \\ p(n) &= n/2\end{aligned}$$

因为每一层比较可以并行进行，而树高是 $\log n$ 。

9. 6.3.3 计算前缀和

9.1 问题定义

给定 n 个元素：

$$\{x_1, x_2, \dots, x_n\}$$

前缀和是：

$$s_i = x_1 * x_2 * \dots * x_i, 1 \leq i \leq n$$

其中 $*$ 可以是：

+

也可以是：

$\times$

所以这里讲的是更一般的前缀运算，也叫 scan。

9.2 串行算法

串行算法是：

$$s_i = s_{i-1} * x_i$$

从左到右依次计算。

时间复杂度：

$$O(n)$$

9.3 并行算法思想

课件使用平衡树做非递归并行算法。

定义：

$$A[i] = x_i$$

辅助数组：

$$\begin{aligned}B[h, j] \\ C[h, j]\end{aligned}$$

其中：

```
h = 0 ~ log n
j = 1 ~ n / 2^h
```

9.4 B 数组作用

$B[h, j]$ 记录由叶到根正向遍历树中各节点的信息。

也就是先做归约，求每个子树的部分和。

可以理解为：

B 保存“这棵子树的总和”

9.5 C 数组作用

$C[h, j]$ 记录由根到叶反向遍历树中各节点的信息。

也就是从根向叶传播前缀信息。

可以理解为：

C 保存“到这个位置为止应继承的前缀”

9.6 $n = 8$ 的例子

第 30 页图展示 $n = 8, p = 8$ 的情况。

9.6.1 正向遍历

正向遍历由叶到根。

底层是：

$A_1, A_2, \dots, A_8$

先两两求和：

```
B11 = A1 * A2
B12 = A3 * A4
B13 = A5 * A6
B14 = A7 * A8
```

再往上一层：

```
B21 = B11 * B12
B22 = B13 * B14
```

最后根节点：

$B_{31} = B_{21} * B_{22}$

这一步得到总和。

9.6.2 反向遍历

反向遍历由根到叶，用来把前缀信息分发给每个位置。

课件图中给出规则：

```
C[h,1] = B[h,1], j = 1
C[h,j] = C[h+1, j/2], j 为偶数
C[h,j] = C[h+1, (j+1)/2] * B[h,j], j 为奇数且 j > 1
```

直观上：

- 左孩子继承父节点已有前缀；
- 右孩子需要加上左兄弟子树的总和；
- 最终叶子层 $C_{01} \sim C_{08}$ 就是每个元素的前缀和。

9.7 前缀和的意义

前缀和是并行算法中非常基础的操作。

它常用于：

- 并行排序；
- 并行压缩；
- 并行编号；
- 图算法；
- 字符串算法；
- 稀疏矩阵计算；
- GPU scan 操作。

10. 6.4 倍增设计技术

10.1 设计思想

倍增设计技术也叫：

pointer jumping
指针跳跃

特别适合处理：

链表
有向树
森林
父指针结构

它的核心思想是：

■ 每一步把处理距离扩大一倍，经过 k 步后即可完成距离为 2^k 的数据计算。

也就是：

第 0 步：看 1 步远
第 1 步：看 2 步远
第 2 步：看 4 步远
第 3 步：看 8 步远
...

所以通常 $O(\log n)$ 步就能完成原来需要 $O(n)$ 步的链式计算。

11. 6.4.2 表序问题

11.1 问题描述

给定一个 n 个元素的链表 L ，求每个元素在链表中的次第号，也叫：

$\text{rank}(k)$

课件中把 $\text{rank}(k)$ 看成：

■ 元素 k 到表尾的距离。

11.2 例子

课件给出链表：

$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f \rightarrow g$

其中：


```
p[a] = b
p[b] = c
p[c] = d
p[d] = e
p[e] = f
p[f] = g
p[g] = g
```

`g` 是表尾，所以指向自己。

初始化距离：

```
r[a] = r[b] = r[c] = r[d] = r[e] = r[f] = 1
r[g] = 0
```

表示每个非尾节点到自己的后继距离为 1。

11.3 第一次指针跳跃

每个节点把后继改成“后继的后继”。

例如：

```
p[a] = c
p[b] = d
p[c] = e
p[d] = f
p[e] = g
p[f] = g
p[g] = g
```

距离同步累加：

```
r[a] = 2
r[b] = 2
...
r[f] = 1
r[g] = 0
```

11.4 第二次指针跳跃

再次跳跃：

```
p[a] = e
p[b] = f
p[c] = g
p[d] = g
...
```

距离变成：

```
r[a] = 4
r[b] = 4
r[c] = 4
r[d] = 3
r[e] = 2
r[f] = 1
r[g] = 0
```

11.5 最终结果

经过足够多轮后，所有节点都直接指向尾节点 `g`：

```
p[a] = p[b] = ... = p[g] = g
```

最终 rank 为：

```
r[a] = 6
r[b] = 5
r[c] = 4
r[d] = 3
r[e] = 2
r[f] = 1
r[g] = 0
```

11.6 表序算法

课件算法 6.10 可以概括为：

1. 并行初始化 $p[k]$ 和 $distance[k]$
2. 执行 $\lceil \log n \rceil$ 次：
对每个 k 并行执行：
 如果 k 的后继不等于 k 的后继的后继：
 $distance[k] = distance[k] + distance[p[k]]$
 $p[k] = p[p[k]]$
3. 对每个 k 并行执行：
 $rank[k] = distance[k]$

11.7 复杂度

课件给出：

```
t(n) = O(log n)
p(n) = n
```

也就是说，用 n 个处理器，可以把链表排名从串行 $O(n)$ 降到并行 $O(\log n)$ 。

12. 6.4.3 求森林的根

12.1 问题描述

给定一组有向树组成的森林 F 。

如果 $\langle i, j \rangle$ 是 F 中一条弧，则：

```
p[i] = j
```

表示 j 是 i 的父亲。

如果 i 是根，则：

```
p[i] = i
```

目标是求每个节点 j 所在树的根：

```
s[j]
```

12.2 算法思想

这和表序问题类似。

每个节点不断把父指针改成祖父指针：

```
p[i] = p[p[i]]
```

这样节点到根的距离每轮大约减半。

经过 $O(\log n)$ 轮后，每个节点都会直接指向根。

12.3 课件例子

课件第 38 页给出一个森林：

- 一棵树以 8 为根；
- 另一棵树以 13 为根。

初始时：

```
p[1] = p[2] = 5
p[3] = p[4] = p[5] = 6
p[6] = p[7] = 8
p[8] = 8
p[9] = 10
p[10] = 11
p[11] = 12
p[12] = 13
p[13] = 13
```

并初始化：

```
s[i] = p[i]
```

12.4 迭代过程

第一次迭代后，节点跳到祖父。

第二次迭代后，大部分节点已经接近根或直接指向根。

第 39 页图展示了第一次和第二次迭代后的森林形态：树的高度迅速降低，节点逐渐直接挂到根节点下面。

12.5 复杂度

课件给出：

```
t(n) = O(log n)
w(n) = O(n log n)
```

这里 $w(n)$ 是总工作量。

每轮对 n 个节点并行更新，执行 $O(\log n)$ 轮，所以总工作量是 $O(n \log n)$ 。

13. 6.5 流水线设计技术

13.1 设计思想

流水线设计技术的核心是：

将算法流程划分成 p 个前后衔接的任务片段，每个任务片段的输出作为下一个任务片段的输入。

也就是说，计算过程被拆成多个阶段：

```
阶段1 → 阶段2 → 阶段3 → ... → 阶段p
```

每个阶段可以由一个处理器或一个处理单元执行。

13.2 关键要求

课件强调：

所有任务片段按同样的速率产生结果。

这点很重要。

如果某一阶段特别慢，它会成为整个流水线的瓶颈。

13.3 评注

课件指出：

- 流水线技术广泛应用在并行处理中；
- 脉动算法 Systolic algorithm 是一种流水线技术。

脉动算法通常用于矩阵运算、信号处理等场景，数据像“脉搏”一样在处理单元之间周期性流动。

14. 6.5.2 5-point DFT 的计算

14.1 问题描述

课件最后讲 5-point DFT 的流水线计算。

5 点 DFT 可以写成：

$$y_k = a_4 \omega^{4k} + a_3 \omega^{3k} + a_2 \omega^{2k} + a_1 \omega^k + a_0$$

其中：

$$k = 0, 1, 2, 3, 4$$

ω 是单位根。

课件第 44 页把五个输出写成：

$$\begin{aligned} y_0 &= a_4 \omega^0 + a_3 \omega^0 + a_2 \omega^0 + a_1 \omega^0 + a_0 \\ y_1 &= a_4 \omega^4 + a_3 \omega^3 + a_2 \omega^2 + a_1 \omega^1 + a_0 \\ y_2 &= a_4 \omega^8 + a_3 \omega^6 + a_2 \omega^4 + a_1 \omega^2 + a_0 \\ y_3 &= a_4 \omega^{12} + a_3 \omega^9 + a_2 \omega^4 + a_1 \omega^2 + a_0 \\ y_4 &= a_4 \omega^{16} + a_3 \omega^6 + a_2 \omega^4 + a_1 \omega^2 + a_0 \end{aligned}$$

更一般地看，就是对每个 k 计算一个关于 ω^k 的多项式。

14.2 Horner 法则

课件使用秦九韶法则，也就是 Horner 法则。

普通多项式：

$$a_4 x^4 + a_3 x^3 + a_2 x^2 + a_1 x + a_0$$

可以改写成：

$$((((a_4 x + a_3) x + a_2) x + a_1) x + a_0)$$

对于 DFT，令：

$$x = \omega^k$$

于是：

$$y_k = (((a_4 \omega^k + a_3) \omega^k + a_2) \omega^k + a_1) \omega^k + a_0$$

课件中写成：

$$\begin{aligned} y_0 &= (((a_4 \omega^0 + a_3) \omega^0 + a_2) \omega^0 + a_1) \omega^0 + a_0 \\ y_1 &= (((a_4 \omega^1 + a_3) \omega^1 + a_2) \omega^1 + a_1) \omega^1 + a_0 \\ y_2 &= (((a_4 \omega^2 + a_3) \omega^2 + a_2) \omega^2 + a_1) \omega^2 + a_0 \\ y_3 &= (((a_4 \omega^3 + a_3) \omega^3 + a_2) \omega^3 + a_1) \omega^3 + a_0 \\ y_4 &= (((a_4 \omega^4 + a_3) \omega^4 + a_2) \omega^4 + a_1) \omega^4 + a_0 \end{aligned}$$

这样每个输出都可以通过相同的流水线结构计算。

14.3 流水线单元

第 45 页左下角画了一个基本流水线单元。

它接收：

```
X_in
Y_in
```

输出：

```
X_out
Y_out
```

计算关系可以理解为：

```
X_out = X_in
Y_out = Y_in · X_in + a
```

也就是每经过一个阶段，就完成 Horner 法则中的一步：

```
当前结果 × x + 当前系数
```

14.4 5-point DFT 流水线结构

第 45 页右侧图展示了 5 个输出 y0 ~ y4 的流水线计算过程。

每一行对应一个输出：

```
y0, y1, y2, y3, y4
```

每一行都经过若干个系数处理阶段：

```
a4 → a3 → a2 → a1 → a0
```

不同的是每一行使用的 ω 幂不同：

```
y0 使用 ω^0 = 1
y1 使用 ω
y2 使用 ω^2
y3 使用 ω^3
y4 使用 ω^4
```

所以各行结构相同，但输入的 ω^k 不同。

14.5 复杂度

课件给出：

```
p(n) = n - 1
t(n) = 2n - 2 = O(n)
```

对于 5-point DFT，就是用流水线方式让多个输出在不同阶段重叠执行。

这里的重点不是说 DFT 最优复杂度是 O(n)，而是说明：

当一个计算可以拆成连续阶段，并且不同数据可以连续进入这些阶段时，就可以用流水线提高吞吐率。

15. 本章五种技术总对比

技术	核心结构	典型例子	主要优势
----	------	------	------

技术	核心结构	典型例子	主要优势
均匀划分	平均分数据	PSRS 排序	简单、负载均衡
方根划分	按 $\sqrt{n}$ 分块	Valiant 归并	递归深度低
对数划分	按 $\log n$ 分块	并行归并排序	子问题规模适中
功能划分	按算法功能分组	m-n 选择	服务特定目标
分治	子问题递归并行	双调归并网络	结构清晰
平衡树	树形归约/传播	最大值、前缀和	$O(\log n)$ 时间
倍增	距离逐步翻倍	表序、森林根	链式结构并行化
流水线	阶段串接	5-point DFT	提高吞吐率

16. 复习重点

16.1 需要掌握的概念

本章最重要的是理解五类设计技术：

划分
分治
平衡树
倍增
流水线

16.2 需要掌握的典型算法

对应例子如下：

均匀划分	→ PSRS排序
方根划分	→ valiant归并
对数划分	→ 对数划分并行归并排序
功能划分	→ (m,n)选择
分治	→ 双调归并网络
平衡树	→ 求最大值、前缀和
倍增	→ 表序问题、求森林根
流水线	→ 5-point DFT

16.3 一句话总结

第六章的核心是：

并行算法设计不是只把循环改成并行循环，而是要根据问题结构选择合适的设计技术：规则数据用划分，递归问题用分治，归约传播用平衡树，链式依赖用倍增，阶段化计算用流水线。
